

Tutorial 2.3 - Upper Confidence Bound (Solutions)

LfBD Team, 2026

October 2026

In this tutorial, we will be covering the Upper Confidence Bound (UCB) method to solve multi-armed bandit problems.

Dataset

For this tutorial, we will work with the Yahoo dataset, which was also used in tutorial 2.1.

```
# reads in full csv of Yahoo dataset
dfYahoo <- read.csv('yahoo_day1_10arms_tiny.csv')[,-c(1,2)]

# selects the two relevant columns from Yahoo dataset; arm shown to user and reward observed
dfYahoo<- dfYahoo %>% select(arm, reward)
dfYahoo$index <- 1:nrow(dfYahoo)
```

UCB: notation

As covered in the lecture, UCB methods decide on the arm to pick using an ‘optimistic’ estimate of the reward - e.g. which arm has the most potential to yield a high reward? This is done by picking the arm that has the highest combination of (1) the estimated reward, $Q_t(a)$, and (2) a bonus that rewards the arm for being underexplored, denoted as $U_t(a)$. The UCB algorithm picks the arm that maximizes the sum of these two factors:

$$a_t = \operatorname{argmax}_{a \in \mathcal{A}} Q_t(a) + c \cdot U_t(a). \quad (1)$$

$$(2)$$

In the general case of the UCB algorithm, $U_t(a) = \sqrt{\frac{\log(t)}{N_t(a)}}$, where $N_t(a)$ is the number of times arm a has been pulled so far.

Task 1: select the arm according to the UCB algorithm based on the data below. Here, the code already provides a dataframe with per arm (1) the average reward, (2) the number of pulls thus far, and (3) an given value for t . Use this dataframe to determine which arm should be pulled if $c = 0.1$. The selected arm should be 8.

```
# set the seed
set.seed(0)

c <- 0.1

#####
# policy_ucb : picks an arm, based on the UCB algorithm for multi - armed bandits
```

```

#
# Arguments :
# df: data.frame with two columns: arm, reward
# c : float, UCB penalty parameter
##
# Output :
# chosen_arm ; integer, index of the arm chosen
####
policy_ucb <- function(df, c){
  # get per item, the average reward and the number of items observed
  dfsummary <- df %>%
    group_by(arm) %>%
    summarise(avg_reward = mean(reward),
              n_pulls= n())

  # the t in this case is simply the total of observations
  t <- sum(dfsummary$n_pulls)

  # TODO: Select an arm based on the UCB criterion (using equation (1))
  ucb_reward <- dfsummary$avg_reward + c*sqrt(log(t)/dfsummary$n_pulls)
  chosen_arm <- which.max(ucb_reward)

  return(chosen_arm)
}

# get the chosen arm from the function
ucb_ichosen_item <- policy_ucb(dfYahoo, c)
print(paste0('The chosen arm is: ', ucb_ichosen_item))

```

UCB: code

We will be simulating the performance of the UCB algorithm on the Yahoo! data, again with the contextual package.

```

####
# sim_ucb: simulates performance of a UCB policy
#
# Arguments:
#
#   df: n x 3 data.frame, with column names "arm", "reward", "index"
#
#   n_before_sim: integer, number of observations (randomly sampled)
#                 before starting the UCB algorithm
#
#   n_sim: integer, number of observations used to simulate the
#          performance of the epsilon-greedy algorithm
#
#   c: float, UCB penalty parameter
#
#   interval: the number of steps after which our arm is updated.
#             For example, interval is 5 means that when an arm

```

```

#           is chosen by our approach, it is deployed for 5 steps.
#
#
#   Output: list with following
#     df_results_of_policy: n_sim x 2 data.frame, with "arm", "reward".
#                           Random sampled rewards for chosen arms
#
#     df_sample_of_policy: data.frame with sample used to evaluate policy.
#
#
###
sim_ucb <- function(df, n_before_sim, n_sim, c, interval=1){

  ## Part 1: create two dataframes, one with data before start of policy, and one with data after

  # define the number of observations of all data available
  n_obs <- nrow(df)

  # Give user a warning: the size of the intended experiment is bigger than the data provided
  if(n_sim > (n_obs - n_before_sim)){
    stop("The indicated size of the experiment is bigger than the data provided - shrink the size ")
  }

  # Next we prepare our dataframes using a function from helper_functions.R
  prepped_data <- prepare_dataframes(df, n_obs, n_before_sim, n_sim)

  # Access individual data frames
  df_results_policy <- prepped_data$df_results_policy
  df_during_policy  <- prepped_data$df_during_policy
  df_results_at_t   <- prepped_data$df_results_at_t

  ## part 2: apply UCB algorithm, updating at interval
  for(i in 1:n_sim){

    # update at interval
    if((i==1) || ((i %% interval)==0)){

      # TODO: pick an arm according to the UCB policy using the function from Task 1
      chosen_arm <- policy_ucb(df_results_at_t,c)
      current_arm <- chosen_arm

    }else{

      # TODO: if not updating, take current arm
      chosen_arm <- current_arm
    }

    # select from the data for experiment the arm chosen
    df_during_policy_arm <- df_during_policy %>%
      filter(arm==chosen_arm)

    # warn the user to increase the dataset or downsize the experiment,

```

```

# in the case that we have sampled all observations from an arm
if(nrow(df_during_policy_arm) == 0){
  print("You have run out of observations from a chosen arm")
  break
}

# randomly sample from this arm and observe the reward
sampled_arm <- sample(1:nrow(df_during_policy_arm), 1)
reward <- df_during_policy_arm$reward[sampled_arm]

# important: remove the sampled observation from the dataset to prevent repeated sampling
index_result <- df_during_policy_arm$index[sampled_arm]
df_during_policy <- df_during_policy %>% filter(index != index_result)

# get a vector of results from chosen arm (arm, reward)
result_policy_i <- c(chosen_arm, reward)

# add to dataframe to save the result
df_results_policy[i,] <- result_policy_i

# TODO: combine to dataframe with all results
df_results_at_t <- rbind(df_results_at_t, result_policy_i)
}

# save results in list
# note: 'index_before_sim' lives inside prepare_dataframes() and is not returned, so
# df[-index_before_sim,] errors here. df_during_policy is that same data: all observations
# except the ones used before the policy started.
results <- list(df_results_of_policy = df_results_policy,
               df_sample_of_policy = df_during_policy)

return(results)
}

```

Task 2: Run 10 simulations and calculate the cumulative reward per simulation (1-10).

```

# number of observations used to simulate
n_sim <- 2500

# The number of simulations
num_simulations <- 10
c <- 0.1

# set the seed
set.seed(0)

# Create an empty dataframe where the results of storing the simulator are stored
df_Yahoo_UCB_01 <- data.frame(matrix(NA, nrow = 1, ncol = 2))
colnames(df_Yahoo_UCB_01) <- c('arm', 'reward')

# Loop to get 10 simulations
for (i in 1:num_simulations){

```

```

# TODO: run UCB simulation using the function built in the previous task
df_Yahoo_UCB_01_temp <- sim_ucb(dfYahoo, n_before_sim=100, n_sim=n_sim, c=c, interval=1)[[1]]

# Append the results to our dataframe
df_Yahoo_UCB_01 <- rbind(df_Yahoo_UCB_01, df_Yahoo_UCB_01_temp)
}

df_Yahoo_UCB_01 <- df_Yahoo_UCB_01[-1,]
df_Yahoo_UCB_01$simulation <- rep(1:num_simulations, each=n_sim)

```

Task 3: repeat the steps of this tutorial, but now for $c = 0.5$.

```

c <- 0.5

# set the seed
set.seed(0)

# Empty dataframe where the results of storing the simulator are stored
df_Yahoo_UCB_05 <- data.frame(matrix(NA, nrow = 1, ncol = 2))
colnames(df_Yahoo_UCB_05) <- c('arm', 'reward')

# Loop over the simulations
for (i in 1:num_simulations){

  # TODO: Run the UCB algorithm for c = 0.5
  df_Yahoo_UCB_05_temp <- sim_ucb(dfYahoo, n_before_sim=100, n_sim=n_sim, c=c, interval=1)[[1]]

  # Append the results to our dataframe to keep track of the results
  df_Yahoo_UCB_05 <- rbind(df_Yahoo_UCB_05, df_Yahoo_UCB_05_temp)
}

df_Yahoo_UCB_05 <- df_Yahoo_UCB_05[-1,]
df_Yahoo_UCB_05$simulation <- rep(1:num_simulations, each=n_sim)

```

Task 3: Make a plot that compares the UCB policy for $c = 0.1$, $c = 0.5$. Compare the policies based on average cumulative reward. Can you conclude which one performs better, and if so why?

Your answer to Task 3 here:

```

# set max observations to create fair comparison across simulations
max_obs <- 2500

df_Yahoo_UCB_01$t <- 1:n_sim
df_history_agg_01 <- df_Yahoo_UCB_01 %>%
  group_by(simulation)%>% # group by simulation
  mutate(cumulative_reward = cumsum(reward))%>% # calculate, per sim, cumulative reward over time
  group_by(t) %>% # group by timestep
  summarise(avg_cumulative_reward = mean(cumulative_reward), # average cumulative reward
            se_cumulative_reward = sd(cumulative_reward, na.rm=TRUE)/sqrt(num_simulations)) %>% # SE +
  mutate(cumulative_reward_lower_CI = avg_cumulative_reward - 1.96*se_cumulative_reward,
         cumulative_reward_upper_CI = avg_cumulative_reward + 1.96*se_cumulative_reward)%>%
  filter(t <= max_obs)

df_Yahoo_UCB_05$t <- 1:n_sim
df_history_agg_05 <- df_Yahoo_UCB_05 %>%

```

```

group_by(simulation)%>% # group by simulation
mutate(cumulative_reward = cumsum(reward))%>% # calculate cumulative reward
group_by(t) %>% # group by timestep t
summarise(avg_cumulative_reward = mean(cumulative_reward), # calculate average cumulative reward
          se_cumulative_reward = sd(cumulative_reward, na.rm=TRUE)/sqrt(num_simulations)) %>% # calculate standard error
mutate(cumulative_reward_lower_CI = avg_cumulative_reward - 1.96*se_cumulative_reward,
       cumulative_reward_upper_CI = avg_cumulative_reward + 1.96*se_cumulative_reward)%>%
filter(t <= max_obs)

# combine the dataframes
df_history_agg_ucb <- bind_rows(df_history_agg_01 %>% mutate(c='0.1'), df_history_agg_05 %>% mutate(c='0.5'))

# TODO: make a plot using ggplot to compare UCB policy for c=0.1 and c=0.5
# 1: A plot that shows only the average cumulative rewards over time using the df_history_agg dataframe
# 2: The plot as defined in (1) together with the 95% confidence interval.

ggplot(data=df_history_agg_ucb, aes(x=t, y=avg_cumulative_reward, color =c)) +
  geom_line(size=1.5)+ # create line
  geom_ribbon(aes(ymin=ifelse(cumulative_reward_lower_CI<0, 0,cumulative_reward_lower_CI) , # create confidence interval
                ymax=cumulative_reward_upper_CI,
                fill = c,
                ),
            alpha=0.1)+
  labs(x = 'Time', y= 'Cumulative Reward ', color = 'c', fill= 'c')+
  theme_bw()+
  theme(text = element_text(size=16))

```

Task 4: Suppose we compare the UCB policy for $c = 0.1$ to an ϵ -greedy policy where $\epsilon = 0.1$, and find that the ϵ -greedy policy works better in this case. Why do you think this might happen?

Answer: Using the ϵ -greedy policy where $\epsilon = 0.1$ scores on average better. This might be because exploring random arms is better than getting ‘stuck’ in arms that might seem to have potential.

Task 5: Brain teaser 3

Suppose the environment changes while the algorithm is running. We use two arms. For the first 700 observations arm 1 succeeds with probability 0.52 and arm 2 with probability 0.48. After those 700 observations the probabilities change to 0.3 for arm 1 and 0.7 for arm 2, and we keep running for 500 more observations.

Answer:

1. Rolling window. This is a simple method in which only the k most recent observations are taken into account when calculating your model. Here, the most important modelling choice is the window size. If it is too small, your algorithm will have very little information to act on, which can severely worsen accuracy. If the window is too large, the algorithm will be too slow to react to changes in the environment. Overall, it can be a useful way to ensure your algorithms stays responsive to the environment, though it is not without drawbacks.
2. Decaying window. This technique assigns progressively less weight to observations further from the most recent observation. In our case, we would down-weight the rewards, making them less important for the algorithm the further in the past they were. The main modelling choice here is the decay parameter, which sets how quickly past observations decay. It also faces the same problems as the rolling window. If the observations decay too rapidly, the algorithm has little information and performs too poorly. If they decay too slowly, the algorithm is too slow to react to changes. This method is slightly different from the rolling window, as the algorithm never truly “forgets”, i.e. assigns weight to zero, observations.

We provide the script below to show off the differences between the methods.

Compare three ways of estimating the average reward of an arm:

- **naive**: the mean over all observations of that arm
- **rolling window**: the mean over the last W observations of that arm
- **decaying window**: a weighted mean in which an observation that is k pulls old gets weight γ^k

Plot the cumulative reward over t for the three estimators, and explain which one adapts to the regime change and why.

Only the estimate of the average reward differs between the three; the UCB exploration bonus is left as it is, so the comparison isolates the effect of the estimator.

```
# two arms, success probabilities before and after the regime change
p_before <- c(0.52,0.48)
p_after  <- c(0.3, 0.7)
n_before <- 700
n_after  <- 500
n_sim    <- n_before + n_after

# settings of the two non-stationary estimators
window <- 200      # rolling window: number of most recent pulls of an arm that are kept
gamma  <- 0.95     # decaying window: weight of an observation one pull older

# exploration parameter of the UCB policy
c <- 0.1

# TODO: write the three estimators. Each takes the rewards of one arm, oldest first,
#       and returns that arm's estimated average reward
estimate_naive <- function(rewards){
  mean(rewards)
}

estimate_rolling <- function(rewards){
  mean(tail(rewards, window))
}

estimate_decaying <- function(rewards){
  age <- rev(seq_along(rewards)) - 1 # the most recent observation has age 0
  sum(gamma^age * rewards) / sum(gamma^age)
}

estimators <- list(naive          = estimate_naive,
                  `rolling window` = estimate_rolling,
                  `decaying window` = estimate_decaying)

# simulate one run: UCB on two arms, with the probabilities swapping after n_before pulls
sim_regime_change <- function(estimator, seed){

  set.seed(seed)

  history <- list(numeric(0), numeric(0)) # rewards observed per arm
  arms    <- integer(n_sim)
  rewards <- integer(n_sim)
```

```

for(i in 1:n_sim){

  # the regime change: after n_before observations the two probabilities swap
  p <- if(i <= n_before) p_before else p_after

  if(i <= 2){
    # pull each arm once so that both have an estimate
    chosen_arm <- i
  }else{
    avg_reward <- sapply(history, estimator)
    n_pulls <- sapply(history, length)
    chosen_arm <- which.max(avg_reward + c*sqrt(log(i)/n_pulls))
  }

  reward <- rbinom(1, 1, p[chosen_arm])

  history[[chosen_arm]] <- c(history[[chosen_arm]], reward)
  arms[i] <- chosen_arm
  rewards[i] <- reward
}

data.frame(t = 1:n_sim, arm = arms, reward = rewards)
}

# TODO: run each estimator a number of times and average the cumulative reward over the runs
n_runs <- 100

df_runs <- bind_rows(lapply(names(estimators), function(nm){
  bind_rows(lapply(1:n_runs, function(r){
    sim <- sim_regime_change(estimators[[nm]], seed = r)
    data.frame(t = sim$t, cumulative_reward = cumsum(sim$reward), estimator = nm)
  })))
}))

df_plot <- df_runs %>%
  group_by(estimator, t) %>%
  summarise(avg_cumulative_reward = mean(cumulative_reward), .groups = 'drop')

# TODO: plot the average cumulative reward over t for the three estimators
ggplot(df_plot, aes(x = t, y = avg_cumulative_reward, colour = estimator)) +
  geom_vline(xintercept = n_before, linetype = 'dashed', colour = 'grey40') +
  annotate('text', x = n_before, y = 0, label = 'probabilities swap',
    hjust = 0, vjust = 0, size = 3.5, colour = 'grey40') +
  geom_line(linewidth = 0.9) +
  labs(x = 'Time', y = 'Cumulative reward', colour = NULL,
    title = paste0('Cumulative reward around a regime change (mean of ', n_runs, ' runs)')) +
  scale_colour_manual(values = c('naive' = 'grey35',
    'rolling window' = '#2c7fb8',
    'decaying window' = '#c0392b')) +
  theme_bw() +
  theme(legend.position = 'top', text = element_text(size = 12))

```

Up until the point where the probabilities change, all policies perform similarly. Note that this is mainly due

to both arms having a similar success chance, along with the fact there are only 2. If there were more arms, the naive algorithm would outperform the other policies, as it can retain all past information. After the probability break, we see that the decaying window pulls ahead of the other algorithms, as it recognizes the probabilities have changed. The rolling window needs a longer time to internalize the change in probabilities, partly due to the relatively long $k = 200$. Note that these results can change depending on the parametrisation of the algorithms. Experiment with the probabilities, window size and gamma decay rate! Are the results what you expected?